

# Online Real Estate System - Java & Spring Boot Annotations

This document provides a comprehensive list of all the Java and Spring Boot annotations used in the **Online Real Estate System** backend, categorized by their purpose, along with an explanation of how they are used in the project.

---

## 1. Core Spring & Configuration Annotations

- **@SpringBootApplication**
    - **Usage:** Marks the main class ( `OnlineRealEstateSystemApplication.java` ). It encapsulates `@Configuration` , `@EnableAutoConfiguration` , and `@ComponentScan` to bootstrap the Spring application context.
  - **@Component**
    - **Usage:** A generic stereotype for any Spring-managed component (e.g., utility classes like `JwtUtil` ).
  - **@Service**
    - **Usage:** Marks classes holding business logic (e.g., `LoanApplicationService` , `PropertyService` ).
  - **@Repository**
    - **Usage:** Marks Data Access Object (DAO) interfaces that interact with the database (e.g., `UserRepository` , `PropertyRepository` ).
  - **@Configuration**
    - **Usage:** Indicates that a class declares one or more `@Bean` methods and may be processed by the Spring container to generate bean definitions (e.g., `SecurityConfig` , `WebSocketConfig` ).
  - **@Bean**
    - **Usage:** Indicates that a method produces a bean to be managed by the Spring container.
  - **@Autowired**
    - **Usage:** Enables automatic dependency injection of Spring beans into classes (e.g., injecting `UserRepository` into `UserService` ).
  - **@EnableAsync & @Async**
    - **Usage:** Enables Spring's asynchronous method execution capability. Used to run background tasks without blocking the main thread.
- 

## 2. Spring Web & REST API Annotations

- **@RestController**
  - **Usage:** A convenience annotation that combines `@Controller` and `@ResponseBody` , ensuring all methods return data (like JSON) rather than view templates.
- **@RequestMapping**
  - **Usage:** Maps web requests to specific controller classes or methods (e.g., `@RequestMapping("/api/loans")` ).
- **@GetMapping** , **@PostMapping** , **@PutMapping** , **@DeleteMapping**

- **Usage:** Specialized routing annotations for mapping HTTP GET, POST, PUT, and DELETE requests respectively.
  - **@RequestParam**
    - **Usage:** Extracts query parameters or form-data from the HTTP request URL.
  - **@PathVariable**
    - **Usage:** Extracts dynamic values directly from the URL path (e.g., `/user/{id}` ).
  - **@RequestBody**
    - **Usage:** Binds the HTTP request body (usually JSON) directly to a Java object representing the payload.
  - **@RequestHeader**
    - **Usage:** Extracts specific HTTP headers from the request, heavily used in this project to extract the `Authorization: Bearer <token>` for JWT verification.
  - **@CrossOrigin**
    - **Usage:** Enables Cross-Origin Resource Sharing (CORS), allowing the React frontend (running on a different port/domain) to communicate with the Spring backend.
- 

### 3. Spring Data JPA & Hibernate Annotations (Database)

- **@Entity**
  - **Usage:** Specifies that the class is a JPA entity mapped to a database table.
- **@Table**
  - **Usage:** Allows specifying the exact name of the database table (e.g., `@Table(name = "users")` ).
- **@Id**
  - **Usage:** Marks a field as the primary key of the entity.
- **@GeneratedValue**
  - **Usage:** Configures the way the primary key is generated (e.g., `GenerationType.IDENTITY` for auto-incrementing integers).
- **@Column**
  - **Usage:** Specifies mapped column details in the database, such as length, uniqueness, or if it is nullable (e.g., `@Column(length = 50)` ).
- **@ManyToOne & @OneToMany**
  - **Usage:** Defines relational mappings between entities. For example, a User can have many Loan Applications ( `@OneToMany` ), but a Loan Application belongs to one User ( `@ManyToOne` ).
- **@JoinColumn**
  - **Usage:** Specifies the foreign key column used when joining an entity association or a collection.
- **@PrePersist**
  - **Usage:** Specifies a callback method that is triggered right before an entity is saved for the first time (used to automatically set `createdAt` timestamps).
- **@UniqueConstraint**
  - **Usage:** Defines a unique constraint at the table level (e.g., ensuring emails are unique).
- **@Transactional**

- **Usage:** Ensures that the execution of a method happens within a database transaction. If the method fails, changes are rolled back.
  - **@Query & @Param**
    - **Usage:** Allows writing custom JPQL or native SQL queries directly within a Repository interface.
- 

## 4. Security & WebSocket Annotations

- **@EnableWebSecurity**
    - **Usage:** Enables Spring Security's web security support and provides the Spring MVC integration (used in `SecurityConfig` ).
  - **@EnableWebSocketMessageBroker**
    - **Usage:** Enables broker-backed messaging over WebSocket using a higher-level messaging sub-protocol (STOMP).
  - **@MessageMapping**
    - **Usage:** Maps a specific message destination to a handler method (similar to `@RequestMapping` but for WebSockets).
  - **@DestinationVariable**
    - **Usage:** Extracts values from the destination template of a WebSocket message.
- 

## 5. Miscellaneous Annotations

- **@JsonIgnore (Jackson)**
  - **Usage:** Prevents a field from being serialized into JSON. Used extensively on bidirectional relationships (like `LoanDocument` back to `LoanApplication` ) to prevent infinite nesting recursion depth errors.
- **@NotNull**
  - **Usage:** Indicates that a variable or parameter cannot be null.
- **@SuppressWarnings**
  - **Usage:** Instructs the compiler to suppress specific warnings.